Integrating Large Language Models into Programming Education: Opportunities, Challenges, and Learning Impacts

Fionn McGoldrick

Abstract—*This literature review looks at how Large Language Models (LLMs) are being used in programming education and what that means for students and teachers. Recent studies show that LLMs can explain code, give quick feedback, and help beginners understand difficult concepts in a more accessible way. They can reduce confusion, speed up problem-solving, and make learning feel more personalised. However, the research also points out real challenges. LLMs can produce incorrect answers, students may become too dependent on them, and there are concerns about plagiarism and reduced practice. Some studies report improved understanding when LLMs are used as support tools, while others show little benefit if students rely on them to generate full solutions. Overall, the review suggests that LLMs work best when used to guide and support learning rather than replace actual coding practice. The goal of this paper is to summarise what current research says, highlight the benefits and risks, and point out what still needs to be explored in the future.*

1 Introduction

Learning to code is one of the most important skills in today's world, but teaching it effectively to large groups of students has always been a major struggle. Teachers often can't give each student the energy and personalised help that they need, and getting instant, helpful feedback on code errors is tough. This has all changed since the sudden appearance of Large Language Models (LLMs). Tools like ChatGPT and GitHub Copilot can write, explain, and fix code with impressive skill. This new technology is shaking up programming classes, creating huge potential for improvement, but also causing confusion and concern.

2 Background and Context

Before diving into the specific impact of Large Language Models (LLMs), it's crucial to understand the long-standing hurdles in programming education that these new tools promise to address. Historically, teaching introductory programming, often designated as CS1, has been notoriously challenging for both educators and students.

2.1 Traditional Obstacles in Education

Learning to program requires students to develop multiple interconnected skills at the same time, which often leads to cognitive overload. Beginners must adopt a form of step by step logical reasoning that differs from everyday thinking, while also understanding abstract concepts such as variables, control flow, and data structures. Because program execution is dynamic but code is static, students often struggle to mentally trace how programs behave. Prior research highlights that certain pedagogical approaches can intensify this overload, suggesting a need for more research informed instructional design to support novice learners [1].

Another persistent challenge is the limited availability of timely and individualised feedback. In traditional classroom settings, students frequently receive feedback long after completing programming tasks, reducing its effectiveness for learning. Debugging skills develop through immediate and iterative practice, yet large class sizes often prevent instructors from providing detailed support. As a result, many students become stuck on minor syntax or logic errors that hinder deeper conceptual understanding.

3 Methodology

This literature review uses a qualitative, document-based methodology focused on analysing peer-reviewed research published between the years 2021 and 2025. Source selection employed a systematic search strategy across four major academic databases: ACM Digital Library, IEEE Xplore, MDPI (Multidisciplinary Digital Publishing Institute), and arXiv. Boolean search operators were used to identify relevant literature, combining terms such as ("Large Language Models" OR "LLM" OR "GPT" OR "Codex") AND ("programming education" OR "computer science education" OR "CS1" OR "introductory programming") AND ("learning outcomes" OR "feedback" OR "adaptive learning").

3.1 Inclusion Criteria

Papers were selected based on predefined inclusion criteria to ensure relevance and quality. Studies published between 2021 and 2025 were considered, focusing on the use of large language models in programming education, including applications such as code generation, explanation, debugging, and automated feedback. Only peer reviewed journal articles, conference papers, and formally archived preprints from recognised venues were included.

Selected studies were analysed using a thematic coding approach, grouping findings into key areas such as LLM applications, benefits and limitations, learning outcomes, and pedagogical considerations. This process produced a representative corpus of empirical and survey based research

reflecting current trends in LLM supported programming education.

3.2 Exlusion Criteria

Sources were excluded based on three criteria: (1) the absence of peer review or formal archival status, which removed blog posts, opinion pieces, and other non-academic commentary; (2) a lack of direct relevance to programming education or the use of LLMs and AI-based tools, including studies focused on general educational technology without a programming component; and (3) insufficient methodological detail, such as anecdotal reports or papers that did not present clear data, evaluation procedures, or analytical frameworks.

Only peer-reviewed or formally archived research was included. Informal articles, editorials, and non-academic commentary were excluded to maintain academic quality. All citations follow `BibLaTeX` formatting for consistency throughout the review.

4 Applications of LLMs in Programming Education

Large Language Models are increasingly used across multiple stages of programming education, supporting tasks such as code generation, explanation, debugging, and automated feedback. Their ability to interpret natural language queries and analyse source code makes them particularly suited to learning environments where timely feedback and iterative practice are essential. The following subsections summarise the most common educational applications of LLMs.

4.1 Code Generation

Large language models such as Codex and GPT 4 can generate syntactically correct and contextually appropriate code from natural language prompts. In programming education, these tools are commonly used to prototype initial solutions, explore alternative approaches, and translate problem descriptions into executable code [3, 7]. Instructors report that this can help beginners overcome initial barriers to starting a task, allowing greater focus on program structure and logic. Generated examples may also be analysed or modified by students, supporting more active engagement with core concepts.

Finnie Ansley et al. examined the impact of OpenAI Codex in introductory programming courses and demonstrated that LLMs can reliably translate natural language prompts into functional code [3]. However, their study emphasises that educational value depends on active student engagement. Learners who analysed and reflected on generated solutions achieved deeper understanding than those who copied code directly, reinforcing the need to treat AI generated code as a learning resource rather than a final answer.

4.2 Code Explanation

Large language models are widely valued in programming education for their ability to explain code in clear and accessible language. This supports novice learners in understanding not only what code does, but why it works and how its components interact. Tools such as ChatGPT can respond to natural language questions by providing concise explanations of control flow, variable usage, and program structure, offering immediate and context specific support during learning [2].

However, this functionality carries notable risks. LLMs may generate plausible but incorrect explanations, which novice programmers may not be able to detect [7]. In addition, frequent reliance on automated explanations can limit the development of independent code reading skills. Although research suggests that using LLMs for explanation is generally less harmful than using them for code generation, the potential for dependency and shallow understanding remains an important pedagogical concern [6].

4.3 Debugging Support

The main educational benefit of LLM supported debugging is reduced feedback delay. Rapid hints can help students resolve errors more quickly and maintain engagement, especially in early programming courses [2]. However, limitations remain significant. LLMs may produce confident but incorrect fixes, and frequent reliance on automated debugging has been linked to weaker independent performance when AI support is removed [7, 6]. As a result, the literature emphasises that debugging tools should be used as guided support rather than authoritative solutions.

4.4 Feedback Generation

LLMs are increasingly used in automated grading and feedback systems to evaluate student code and generate personalised comments on errors, structure, and code quality. The BeGrading framework, for example, demonstrates how LLMs can deliver detailed, context-aware feedback at scale, providing students with immediate guidance that would be difficult to achieve manually in large classes [12]. This rapid feedback can help learners identify mistakes earlier and support iterative improvement during practice.

These applications are transforming how instructors manage workload and how students interact with code. However, the effectiveness of each depends on the context of use and the level of guidance provided to learners [7, 2].

5 Benefits and Opportunities of LLMs in Programming Education

The integration of Large Language Models (LLMs) into the pedagogical landscape of computer science offers several transformative opportunities. Research suggests that these tools can address long-standing challenges in programming education, such as delayed feedback, high cognitive load, and the lack of individualised support [11, 10].

5.1 Personalized and Adaptive Learning Pathways

One of the most significant opportunities presented by large language models is the creation of intelligent and adaptive learning environments. As highlighted by Gyöngyörű (2024), AI based adaptive learning systems play a critical

role in modern digital education by delivering efficient and highly personalised learning experiences [8]. These systems are capable of monitoring student learning progress in real time, maintaining learner motivation by dynamically adjusting task difficulty and providing intelligent learning pathways tailored to individual needs.

This personalisation is particularly beneficial in computer science education, where students often progress at different rates. Large language models can act as a bridge between traditional curriculum delivery and individualised instruction by offering on demand support that closely resembles the scaffolding provided by a human tutor [7, 9]. By generating personalised study materials and adaptive differentiation strategies, educators can better support both high performing and struggling students, helping to sustain engagement and improve learning outcomes [2, 3].

5.2 Equity, Accessibility, and Scalable Support

Large language models offer an opportunity to increase equality of access in programming education by providing continuous instructional support. Studies show that LLM based systems can function as tutor like assistants, delivering explanations and guidance on demand and reducing disparities between students with access to personal mentoring and those in large or resource constrained learning environments [7, 2]. This availability is particularly valuable in introductory programming courses, where timely support can improve retention.

Research on adaptive learning systems further indicates that such tools can benefit socially and academically disadvantaged students. Gyöngyörű (2024) demonstrates that AI based adaptive environments can identify knowledge gaps early and provide targeted support before difficulties compound, contributing to improved learner confidence and reduced early failure rates [8]. In addition, automating routine assessment tasks allows educators to focus on higher level instructional activities such as conceptual mentoring and problem solving, while maintaining assessment consistency [7].

6 Challenges and Limitations

While LLMs present many promising applications in programming education, they also introduce notable challenges that must be addressed to ensure effective and ethical use.

6.1 Over-reliance and Reduced Problem-Solving Skills

A growing concern in the literature is that students may become over reliant on large language models, using them as substitutes for independent reasoning rather than as learning supports. Empirical evidence indicates that frequent and unguided LLM use is associated with reduced problem solving ability and weaker learning outcomes in programming education.

Jošt et al. (2024) report a clear negative correlation between LLM usage frequency and final course grades, with the strongest negative effects linked to reliance on LLMs for code generation [6]. Extensive use for debugging was also associated with poorer performance, while using LLMs primarily for explanation showed the least harmful impact. These findings suggest that sustained dependence can accumulate over time rather than producing isolated learning effects.

Related concerns are raised by Kasneci et al. (2023), who warn that the confident and fluent outputs of LLMs may encourage superficial acceptance of solutions without sufficient verification [7]. Taken together, the literature indicates that over reliance on LLMs is a significant and growing educational challenge, highlighting the need for clear pedagogical guidance to preserve the development of analytical reasoning skills.

6.2 Accuracy and Hallucination

Despite their advanced capabilities, large language models are not consistently reliable and may generate outputs that appear coherent and authoritative but are factually incorrect or logically flawed, a phenomenon commonly referred to as hallucination [7]. In programming contexts, this can manifest as subtly incorrect code, misleading explanations, or incomplete solutions that nonetheless appear plausible. When learners lack sufficient domain knowledge, they may be unable to identify these errors, leading to confusion and the reinforcement of misconceptions rather than conceptual understanding [2]. This risk is particularly pronounced in introductory courses, where students are still developing foundational reasoning skills.

6.3 Academic Integrity

The ability of LLMs to produce entire assignments raises ethical concerns around plagiarism and originality. Without clear guidelines, students may submit AI-generated solutions as their own work [10]. This has prompted educators to reconsider how assessments are structured and to implement new forms of AI usage policies and detection methods.

6.4 Bias and Ethical Concerns

Large language models are trained on extensive and heterogeneous datasets that may contain embedded societal biases or stereotypes. As highlighted by Kasneci et al. (2023), these biases can be reproduced or amplified in generated outputs, potentially exposing learners to skewed or inappropriate perspectives [7]. In educational contexts, such biases pose ethical concerns, particularly when LLMs are deployed at scale without sufficient oversight or content filtering. This raises questions about fairness, inclusivity, and the responsibility of institutions to ensure safe and equitable learning environments.

6.5 Lack of Transparency and Explainability

LLMs often provide answers without a transparent reasoning process, making it difficult for both students and instructors to understand the basis of a given output. This lack of explainability may limit their usefulness in contexts where interpretability is essential [11].

6.6 Technical and Pedagogical Constraints

There are practical barriers to using LLMs effectively. These include access to computing infrastructure, integration with learning management systems, and the need for teacher training [4]. Pedagogically, it is still unclear how best to scaffold student interaction with LLMs to ensure learning gains rather than shortcuts.

Despite their promise, LLMs must be thoughtfully integrated into education systems to avoid undermining foundational skills and academic integrity. Many of these limitations can be addressed through appropriate policy design, instructional framing, and continuous evaluation of their educational impact.

7 Impact on Learning Outcomes

Understanding how large language models influence student learning is essential for evaluating their role in programming education. The literature presents mixed but informative findings, indicating that LLMs can improve certain learning outcomes when used appropriately, while also posing risks to deeper skill development when relied upon excessively.

7.1 Academic Performance and Skill Development

Several studies report short-term performance improvements associated with LLM use, particularly for lower-level tasks such as syntax correction, error identification, and basic algorithm implementation. A meta-analysis by Wang and Fan found that AI-assisted tools can improve immediate task completion and correctness in introductory programming contexts [11]. Similarly, automated feedback systems have been shown to help students identify mistakes more quickly, supporting iterative improvement during practice.

However, evidence also highlights important limitations. Jošt et al. report a negative correlation between frequent LLM use and final course grades, particularly when students rely on LLMs for full solution generation or extensive debugging [6]. Their findings suggest that delegating core reasoning tasks to AI tools reduces engagement with underlying programming concepts, leading to weaker problem-solving skills and lower assessment performance. These results indicate that performance gains are highly dependent on usage patterns, with unstructured reliance producing poorer learning outcomes.

7.2 Engagement, Retention, and Learning Behaviours

LLMs have also been associated with increased student engagement, particularly in early programming courses. Conversational interfaces such as ChatGPT lower the barrier to asking questions and seeking clarification, which can reduce frustration and support persistence [2]. This is especially relevant in large or asynchronous learning environments, where access to instructor support is limited. Studies report improved retention and sustained participation when students can obtain immediate explanations or feedback during practice [4].

At the same time, the literature cautions that increased engagement does not necessarily translate to deeper learning. When students accept LLM-generated outputs without reflection or verification, learning may remain superficial. This reinforces the importance of instructional design that encourages active engagement, explanation, and justification rather than passive consumption of AI-generated content.

7.3 Assessment Design and Educational Implications

The widespread availability of LLMs has significant implications for how learning outcomes are assessed. Traditional take-home programming assignments and isolated coding tasks are increasingly vulnerable to AI-assisted completion, limiting their effectiveness as measures of student understanding [6]. In response, several studies advocate for assessment designs that emphasise reasoning processes, code comprehension, and reflection rather than final outputs alone.

Suggested approaches include oral examinations, code walkthroughs, reflective explanations, and in-class assessments that require students to articulate their problem-solving strategies [10]. These methods aim to preserve assessment validity while aligning evaluation with higher-order learning objectives. The literature suggests that such assessment strategies not only reduce inappropriate AI reliance, but also promote deeper engagement with programming concepts.

7.4 Equity and Access Considerations

Equity remains a critical factor in evaluating the impact of LLMs on learning outcomes. While these tools can broaden access to instructional support and personalised feedback, uneven availability of AI tools and supporting infrastructure may exacerbate existing educational inequalities [8]. Students with greater access to LLMs may benefit disproportionately unless institutions account for access disparities in assessment and instructional design.

Research therefore emphasises that learning outcomes associated with LLM use are shaped not only by technical capability, but also by contextual factors such as access, guidance, and assessment practices. Ensuring equitable deployment is essential to realising the potential benefits of LLM-supported learning without reinforcing structural disadvantage.

Overall, the literature indicates that LLMs can positively influence learning outcomes when integrated within well-designed pedagogical and assessment frameworks. Their impact is not inherent, but contingent on how they are used, what tasks they support, and how student engagement with AI-generated content is structured.

8 Future Research Directions

Although large language models are increasingly used in programming education, key aspects of their impact remain insufficiently understood. Existing research largely focuses on short term performance gains, highlighting a clear gap in evidence on the long term effects of sustained LLM use on conceptual understanding, knowledge retention, and independent problem solving [6, 5]. Longitudinal studies are needed to determine whether early benefits persist once AI support is removed.

Another important gap concerns the populations and contexts studied. Much of the current evidence is drawn from introductory programming courses with novice learners, offering limited insight into how LLMs affect advanced students or professional developers. Future work should examine whether LLM supported learning transfers to industry relevant problem solving and real world programming tasks [7].

Finally, further research is required to understand how LLMs can be integrated reliably and equitably into educational settings. This includes investigating effective scaffolding strategies, improving system reliability, and addressing persistent concerns around bias, misinformation, and unequal access [8, 11]. Overall, future research should focus on how, when, and for whom LLMs support meaningful and transferable learning in programming education.

9 Conclusion

Large language models are increasingly influencing how programming is taught and learned by supporting code generation, explanation, debugging, and automated feedback. The literature reviewed shows that LLMs can reduce feedback delays, improve access to instructional support, and support adaptive and self paced learning, particularly for novice programmers.

However, their educational value depends strongly on how they are used. Evidence indicates that LLMs are most effective as supplementary learning aids rather than replacements for independent problem solving. Code explanation and guided debugging can support conceptual understanding when students actively engage with outputs, while unstructured reliance on code generation or automated debugging is associated with weaker learning outcomes. This highlights the importance of framing AI generated outputs as resources for analysis and reflection rather than final solutions.

The literature also identifies key challenges. Over reliance, academic integrity concerns, hallucination, and bias show that unguided LLM use can undermine core learning objectives. As a result, assessment design must prioritise reasoning and process, and students must be supported in developing AI literacy skills.

Overall, LLMs offer significant potential to support programming education when integrated within well designed pedagogical frameworks. Their impact depends not only on technical capability, but on instructional context, access, and guidance, with future progress relying on continued research into effective integration and long term learning effects.

References

[1] Rachel Cardell-Oliver. "How Students Experience Learning to Program". In: *Education Research and Perspectives* 41 (2014), pp. 196–216. ISSN: 0311-2543.

[2] Fitsum Gizachew Deriba, Ismaila Temitayo Sanusi, and Amos Oyelere. "Enhancing Computer Programming Education Using ChatGPT: A Mini Review". In: *Proceedings of the 23rd Koli Calling Conference* (2023). DOI: 10.1145/3631802.3631848. URL: https://doi.org/10.1145/3631802.3631848.

[3] James Finnie-Ansley et al. "The Robots Are Coming: Exploring the Implications of OpenAI Codex on Introductory Programming". In: *Australasian Computing Education Conference (ACE)*. 2022, pp. 10–19. DOI: 10.1145/3511861.3511863. URL: https://doi.org/10.1145/3511861.3511863.

[4] Ilie Gligorea et al. "Adaptive Learning Using Artificial Intelligence in e-Learning: A Literature Review". In: *Education Sciences* 13.12 (2023). DOI: 10.3390/educsci13121216. URL: https://www.mdpi.com/2227-7102/13/12/1216.

[5] Sven Jacobs and Steffen Jaschke. "Evaluating the Application of Large Language Models to Generate Feedback in Programming Education". In: *2024 IEEE Global Engineering Education Conference (EDUCON)*. IEEE, 2024, pp. 1–5. DOI: 10.1109/EDUCON60312.2024.10578838. URL: http://dx.doi.org/10.1109/EDUCON60312.2024.10578838.

[6] Gregor Jošt, Viktor Taneski, and Sašo Karakatič. "The Impact of Large Language Models on Programming Education and Student Learning Outcomes". In: *Applied Sciences* 14.10 (2024). DOI: 10.3390/app14104115. URL: https://www.mdpi.com/2076-3417/14/10/4115.

[7] Enkelejda Kasneci et al. "ChatGPT for Good? On Opportunities and Challenges of Large Language Models in Education". In: *Learning and Individual Differences* 103 (2023), p. 102274. DOI: 10.1016/j.lindif.2023.102274. URL: https://doi.org/10.1016/j.lindif.2023.102274.

[8] Jozsef Katona and Klara Ida Katonane Gyonyoru. "AI-based Adaptive Programming Education for Socially Disadvantaged Students: Bridging the Digital Divide". In: *TechTrends* 69.5 (2025), pp. 925–942. DOI: 10.1007/s11528-025-01088-8. URL: https://doi.org/10.1007/s11528-025-01088-8.

[9] Juho Leinonen et al. "Using Large Language Models to Enhance Programming Error Messages". In: *arXiv preprint* (2022). eprint: 2210.11630. URL: https://arxiv.org/abs/2210.11630.

[10] Nishat Raihan et al. "Large Language Models in Computer Science Education: A Systematic Literature Review". In: *Proceedings of the 56th ACM Technical Symposium on Computer Science Education V. 1*. New York, NY, USA: Association for Computing Machinery, 2025, pp. 938–944. DOI: 10.1145/3641554.3701863. URL: https://doi.org/10.1145/3641554.3701863.

[11] Shen Wang et al. *Large Language Models for Education: A Survey and Outlook*. 2024. arXiv: 2403.18105 [cs.CL]. URL: https://arxiv.org/abs/2403.18105.

[12] Mina Yousef et al. "BeGrading: Large Language Models for Enhanced Feedback in Programming Education". In: *Neural Computing and Applications* 37 (2025), pp. 1027–1040. DOI: 10.1007/s00521-024-10449-y. URL: https://doi.org/10.1007/s00521-024-10449-y.